> A diagram showing a queue with jobs across three time periods. The first shows two CompleteOrderJob jobs queued, with order #1 ready for processing. The second time period shows the state of the queue after processing the first job. We see that behind the CompleteOrderJob for order #2 is a SendOrderNotificationEmailJob and a RequestOrderFulfillmentJob. The third time period shows that the third order job was created while the CompleteOrderJob for order #2 is behind the two jobs created in the previous period.

There are two constraints on how long it takes for the user to get confirmation that the system has their order: the number of CompleteOrderJob jobs that can be processed at once, and the number of other jobs queued ahead of any new CompleteOrderJob. In both cases, the user waits for something irrelevant to their order's creation.

Sidekiq provides two relatively easy-to-try mechanisms for addressing this: concurrency and priority queues.

Use Queues to Prioritize Jobs

Unlike CompleteOrderJob, the results of the other two jobs aren't time-critical. There's no reason to require a user to wait in order to send notification emails or request fulfillment. If the CompleteOrderJob jobs could be given priority, that would help this issue.

Sidekiq supports this. You can tell Sidekiq to process all jobs on one queue before any job in another queue is processed. You can then queue CompleteOrderJobs to the higher priority queue, thus prioritizing those jobs. If you did that, the following diagram shows how the system would behave over time. Note that the third CompleteOrderJob no longer has to wait for all the jobs, and now just needs to wait for the second CompleteOrderJob before it can be processed.